PRESS Replication Guide

Trey Elder Jonathan Pipping

HALO 2026 Hackathon

1 Replication Scope

This document is the replication-focused companion to the submission writeup. It is for teams who want to reproduce PRESS from the HALO raw data. It focuses on the method itself: forecheck sequence construction, feature engineering, model fitting, and player-credit allocation.

If this document and the code disagree, follow the code. In practice, `scripts/01_forechecks.py` defines the forecheck sample, `scripts/02_features.py` defines the feature set, and `scripts/05_modeling.py` defines the hybrid model and player-credit allocation.

The implementation described below matches the current code in:

- `scripts/01_forechecks.py`
- `scripts/02_features.py`
- `scripts/05_modeling.py`
- `scripts/07_evaluation.py`

In the current repo state, the processed artifacts contain:

- **28,660** forecheck sequences
- **10,242** sequence successes
- **18,418** sequence failures
- **137,465** hazard rows across **27,329** sequences with usable tracking frames

2 Problem Framing

PRESS estimates how much an individual forechecker increases the probability that a dump-in forecheck ends in a defensive-zone turnover before the defending team exits. The main challenge is that forecheck impact is spread across players and across time:

- the puck recoverer is often not the player who created the pressure state;
- forecheck success depends on both the *start configuration* and the *subsequent execution*;

- tracking coverage is incomplete, so the method must be robust to missing player positions;
- player credit must conserve to the observed team-level outcome while still being based on counterfactual player replacement.

We handle that with a four-part pipeline:

1. build forecheck sequences from event data;
2. compute per-frame hazard features from event and tracking data;
3. fit a hybrid start-plus-execution probability model;
4. allocate sequence value to players via slot-level counterfactual replacement.

3 Data and Preprocessing

3.1 Source tables

The method uses five raw parquet files in `data/raw/`:

- `events.parquet`: event stream, including event type, coordinates, actor, team, sequence, and stint metadata;
- `tracking.parquet`: player positions and velocities at each tracked event;
- `stints.parquet`: on-ice players and manpower state by stint;
- `players.parquet`: player identity and position;
- `games.parquet`: home/away metadata.

The main join keys are `game_id`, `sl_event_id`, `player_id`, and `game_stint`.

3.2 Forecheck sequence construction

Sequence construction is fully specified in `01_forechecks.py`. The pipeline builds each forecheck in six steps.

Step 1: identify the forcing event. Find all `dumpin` events. The dumping team is the *pressing team* and the opposing team is the *defending team*.

Step 2: identify the start of forecheck pressure. Within the same `game_id` and `sequence_id`, locate the first successful `lpr` by the defending team after the dump-in where `detail` is in

$$\{\texttt{opdump}, \texttt{opdumpcontested}, \texttt{hipresopdump}, \texttt{hipresopdumpcontested}, \texttt{contested}\}.$$

This `lpr` is the start row of the forecheck. Its event coordinates are stored as `puck_x_at_start` and `puck_y_at_start`.

Step 3: scan forward to the first terminal event. Starting immediately after the start row, scan events in event-id order until the first row that satisfies one of:

- **stoppage:** `whistle`, `goal`, `icing`, `offside`, or `penalty`;
- **zone exit:** the puck crosses the blue-line threshold used in code;
- **pressing-team possession:** the pressing team records a possession event before exit.

The code treats the following as possession events:

$$\{\texttt{lpr}, \texttt{reception}, \texttt{carry}, \texttt{pass}, \texttt{dumpin}, \texttt{shot}\}.$$

Step 4: determine outcome. The blue-line threshold is `BLUE_LINE = 25`. Before orientation normalization, zone exit is detected when:

- the forecheck starts on the negative-x side and a later event satisfies $x > -25$, or
- the forecheck starts on the positive-x side and a later event satisfies $x < 25$.

The initial outcome rule is:

- stoppage or zone exit $\Rightarrow$ failure;
- pressing-team possession before either $\Rightarrow$ success.

Penalty rows override that default:

- penalty on the pressing team $\Rightarrow$ failure;
- penalty on the defending team $\Rightarrow$ success.

Step 5: drop period-end whistles. If the terminal event is a whistle in the last second of the period, drop the sequence rather than scoring it as a success or failure.

Step 6: normalize orientation. For sequences with `puck_x_at_start < -25`, the code negates x and y for sequence-level puck coordinates, event coordinates, tracking coordinates, and tracked velocities. After this transform, all forechecks are represented in the same attacking orientation.

3.3 Sequence outputs

`01_forechecks.py` writes:

- `data/processed/forechecks.parquet`
- `data/processed/forecheck_events.parquet`
- `data/processed/forecheck_tracking.parquet`

In the current build, terminal event counts in `forechecks.parquet` are:

- zone exit: 17,259

- lpr: 10,187
- icing: 553
- whistle: 385
- penalty: 260
- offside: 12
- possession: 2
- goal: 2

4 Analytical Approach

4.1 Hazard-row construction

`02_features.py` converts each tracked event within a forecheck into one model row. Sequences with no usable tracking frames do not appear in `hazard_features.parquet`. That is why the hazard table spans 27,329 sequences rather than all 28,660 forechecks.

For each distinct `(fc_sequence_id, sl_event_id)` pair with tracking:

- `time_since_start_s` is computed from period-adjusted event clock;
- `event_t = 1` only on the terminal success row;
- `terminal_failure_t = 1` only on the terminal failure row;
- all earlier rows have both indicators equal to 0 and are treated as "ongoing".

This yields the three-class target used later:

$$Y_t \in \{0, 1, 2\}$$

with 0 = ongoing, 1 = terminal success, and 2 = terminal failure.

4.2 Carrier and slot features

At each frame, the primary event actor is used as the puck-carrier proxy. The forechecking skaters are the tracked pressing-team skaters, filtered to non-goalies, ranked by distance to the carrier, and assigned slots F1–F5.

For slot i, the main carrier-pressure features are:

$$r_i = \|x_i - x_{\text{carrier}}\|_2 \,,$$
$$v_{r,i}^{\text{carrier}} = v_i^\top \frac{x_{\text{carrier}} - x_i}{\|x_{\text{carrier}} - x_i\|_2}.$$

The code also stores the normalized bearing from carrier to forechecker as

$$(\cos\theta_i, \sin\theta_i),$$

implemented as the x and y components of the slot's unit direction relative to the carrier.

For F2–F5, the code adds nearest-opponent containment features:

$$r_i^{\text{nearestOpp}} = \min_j \left\| x_i - x_j^{\text{opp}} \right\|_2,$$
$$v_{r,i}^{\text{nearestOpp}} = v_i^\top \frac{x_{j^\star}^{\text{opp}} - x_i}{\left\| x_{j^\star}^{\text{opp}} - x_i \right\|_2}.$$

4.3 Lane-blocking proxy

The exit-lane model is implemented directly in `02_features.py`. For each potential pass from the puck carrier to a teammate, the code:

1. define the pass segment from passer a to receiver b;
2. project each forechecker onto that segment;
3. compute the closest point on the lane and the defender's perpendicular distance to it;
4. compare defender arrival time to puck arrival time.

The constants used by the current implementation are:

- intercept radius: `LANE_RADIUS_FT = 3.0`
- assumed puck speed: `PUCK_SPEED_FTPS = 50.0`
- defender floor speed: `DEFENDER_FLOOR_SPEED_FTPS = 8.0`
- base defender reaction time: `DEFENDER_REACTION_S = 0.10`
- center-lane threshold: `|y| <= 15.0`

For a lane, the code computes

$$\text{margin} = t_{\text{defender}} - t_{\text{puck}}.$$

If the margin is negative, the lane is treated as blocked. The forechecker with the most negative margin is treated as the active blocker for that lane. That blocker receives slot-level severity summaries:

- `F{i}_block_severity`
- `F{i}_block_center_severity`

Frame-level lane aggregates are:

- `outlet_candidate_count`

- `unblocked_outlet_count`
- `center_open`
- `min_unblocked_outlet_dist`

with `min_unblocked_outlet_dist` filled to 200.0 if no lane is open.

4.4 Sequence controls

Sequence-level controls are merged from `stints.parquet` and `games.parquet` using the start-event stint:

- `manpower_state` from `n_home_skaters` and `n_away_skaters`
- `pressing_is_home`
- `score_diff_bin` in {`trailing`, `tied`, `leading`}
- `puck_start_x`, `puck_start_y`

4.5 Preprocessing before modeling

Shared preprocessing lives in `scripts/_preprocess.py`. Before modeling, the pipeline:

- add `F1_imputed` through `F5_imputed` to indicate slot-level missingness;
- add `log_time_since_start_s` and `sqrt_time_since_start_s`;
- mean-impute and standardize slot numerics;
- median-impute and standardize non-slot numerics;
- represent `time_since_start_s` with a cubic spline basis (`n_knots = 5`, `degree = 3`, `include_bias = False`);
- most-frequent impute and one-hot encode categorical controls.

4.6 Hybrid probability model

PRESS splits the forecheck into a start component and an execution component.

Start model. The start model predicts

$$p_0 = P(\text{sequence success} \mid \text{start-frame features}).$$

Training uses only the first hazard row per sequence. In the current implementation, the start model is an XGBoost binary classifier with isotonic calibration.

Execution hazard model. The execution model uses all non-start rows and predicts the three-class outcome at each frame:

$$P(Y_t = 0), \quad P(Y_t = 1), \quad P(Y_t = 2).$$

In the current implementation, the execution model is XGBoost with grouped splitting by `fc_sequence_id` and isotonic calibration.

Cumulative incidence calculation. Given row-level predicted hazards

$$h_s(t) = P(Y_t = 1), \qquad h_f(t) = P(Y_t = 2),$$

the code computes execution-phase cumulative incidence using the recursion

$$\begin{aligned} S_0 &= 1, \\ \mathrm{CIF}_s &\leftarrow \mathrm{CIF}_s + h_s(t) S_{t-1}, \\ \mathrm{CIF}_f &\leftarrow \mathrm{CIF}_f + h_f(t) S_{t-1}, \\ S_t &= S_{t-1}\left(1 - h_s(t) - h_f(t)\right), \end{aligned}$$

iterated in event order over non-start rows within a sequence.

The execution value used later is

$$v_{\text{exec}} = \mathrm{CIF}_s - \mathrm{CIF}_f.$$

4.7 Counterfactual slot replacement

Player attribution is implemented in `05_modeling.py`. The core idea is to replace observed slot features with "ghost" versions sampled conditionally on the rest of the frame context.

Sentinel fill for missing slots. Before any replacement, missing slots are filled with conservative uninvolved values:

- distance features set to 80 ft;
- closing speeds set to 0;
- angle features set to neutral direction;
- blocking severities set to 0.

Slot predictors. For each slot F1–F5, a `RandomForestRegressor` is fit to predict that slot's feature vector from the remaining columns. The current code requires at least 100 fully observed rows to fit a slot model. If there are at least 30 start rows or 30 execution rows, separate ghost samplers are fit for those phases. Otherwise, the pooled model is reused.

Ghost sampler. The default option is `-ghost-method rfcde`. For a given context, the sampler:

1. selects a tree;
2. finds the leaf reached by the context;

3. samples one historical slot-feature vector from training rows in that leaf.

4.8 Sequence-level value decomposition

For sequence j with outcome $y_j \in \{0, 1\}$ and start probability p_{0j}, define

$$\bar{p} = \frac{1}{N} \sum_{j=1}^{N} y_j.$$

Then the team-level value is split into:

$$\begin{aligned} \text{Positioning}_j &= p_{0j} - \bar{p}, \\ \text{Execution}_j &= y_j - p_{0j}. \end{aligned}$$

By construction,

$$\text{Positioning}_j + \text{Execution}_j = y_j - \bar{p}.$$

4.9 Slot deltas and credit mapping

For each slot s, the code replaces only that slot with ghost draws and recomputes both the start probability and execution value. With B ghost draws, the raw slot deltas are:

$$\begin{aligned} \Delta^{(s)}_{\text{pos},j} &= \frac{1}{B} \sum_{b=1}^{B} \left(p^{\text{base}}_{0j} - p^{\text{cf},s,b}_{0j} \right), \\ \Delta^{(s)}_{\text{exec},j} &= \frac{1}{B} \sum_{b=1}^{B} \left(v^{\text{base}}_{\text{exec},j} - v^{\text{cf},s,b}_{\text{exec},j} \right). \end{aligned}$$

The current implementation uses `ghost_draws = 10` and `weighting_mode = signed_projected`. Under that rule, raw slot deltas are projected onto the exact sequence total:

$$c_{s,j} = \Delta_{s,j} + \frac{T_j - \sum_r \Delta_{r,j}}{K},$$

where T_j is the sequence total to be allocated and K is the number of modeled slots. This is the minimum-L_2 correction that enforces conservation.

4.10 Reallocation and player-level aggregation

Two implementation details matter for replication:

Empty-slot reallocation. If a sequence has fewer active slots than the modeled maximum, credit initially assigned to empty slots is redistributed across filled slots using the same sign-aware logic. If all slots are empty, the credit is split evenly among pressing-team participants identified from stint data at the terminal event.

Stint attribution within a slot. Slot occupants can change within a sequence. The code constructs within-sequence stints for each slot:

- positioning credit is assigned to the stint containing the start row, or the first stint if the start row is absent;
- execution credit is split across slot stints in proportion to the number of non-start tracked rows they occupy;
- if a slot has no tracked execution rows, execution credit is split evenly across its stints.

Finally, player totals are aggregated across sequences into:

- `pos_total`
- `exec_total`
- `press_total = pos_total + exec_total`
- `press_per_forecheck = press_total / n_forechecks`

5 Implementation Steps

5.1 Environment

From repository root:

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

5.2 End-to-end pipeline

To rebuild the main artifact from raw data:

```
python scripts/01_forechecks.py
python scripts/02_features.py
python scripts/03_simple-attribution.py
python scripts/05_modeling.py
python scripts/06_ranking.py
python scripts/07_evaluation.py
python scripts/_visuals.py
```

The repository also provides:

```
./scripts/run_full_pipeline.sh
```

5.3 Expected outputs

The main files a replication team should check are:

- `data/processed/forechecks.parquet`

- `data/processed/hazard_features.parquet`
- `data/processed/player_press.parquet`
- `data/results/participation.csv`
- `data/results/distance.csv`
- `data/results/modeling.csv`
- `data/results/model_summary.csv`
- `data/results/ranking.csv`
- `plots/calibration_start.png`
- `plots/calibration_hazard.png`

6 Validation and Diagnostics

6.1 Model diagnostics

For the current checked-in run, `data/results/model_summary.csv` reports:

- hazard model: XGBoost
- ghost method: RFCDE
- allocation method: ghost shares with `signed_projected` weighting
- ghost draws: 10
- execution-row log loss: 0.1338

The same file also reports conservation checks. Raw slot deltas do *not* sum perfectly to the sequence total before projection, which is expected. After allocation and stint assignment, the residuals in the current run are effectively zero up to numerical precision.

6.2 Sanity checks for independent replication

If another team reimplements the method, the first checks should be:

1. sequence counts match the expected order of magnitude and success rate;
2. hazard rows are fewer than event rows because only tracked frames are retained;
3. start-model probabilities are well calibrated on held-out sequences;
4. execution hazards satisfy $h_s(t) + h_f(t) \leq 1$ row by row after calibration;
5. player credits sum back to the sequence totals up to floating-point tolerance.

7 Assumptions and Failure Modes

- **Tracking sparsity.** Hazard modeling is limited to frames with tracking, and full-sequence exclusion occurs when no tracked frames are available.
- **Carrier proxy.** The event actor is treated as the puck carrier. This is practical, but it can be wrong on some rows.
- **Pass-lane simplification.** The lane-blocking feature is a geometric intercept proxy, not a physics model of actual passing decisions.
- **Slot abstraction.** Credit is assigned to F1–F5 roles rather than directly to free-moving players, then mapped back through stint logic.
- **Counterfactual support.** Ghost replacement depends on historical training support in random-forest leaves; unusual contexts will be noisier.
- **System dependence.** The method attributes value to the observed players within the observed structure; it does not fully separate player talent from team tactical deployment.

8 Reproducibility

The codebase, commands, and current artifacts are available in this repository and in the project GitHub repo at `https://github.com/WhartonSABI/halo2026`. A team replicating PRESS should treat `scripts/01_forechecks.py`, `02_features.py`, and `05_modeling.py` as the final specification, because small implementation choices in those files can materially affect the resulting player rankings.